

Tymbolica

Exact symbolic computation inside Typst

User manual · version 0.1.0

Parse Typst mathematics, transform it with Symbolica's exact algebra engine,
and place the result directly back into a document.

[Repository](#) · [MIT license](#) · [Symbolica guide](#) · [Symbolica license](#)

Contents

1	Start here	3
1.1	Installation	3
1.2	Choosing an operation	3
2	Core concepts	4
2.1	Expressions are opaque payloads	4
2.2	math, atom, var, and wild	4
2.3	Default functions and custom engines	4
2.4	Systems and return ordering	5
3	Worked mathematical guides	5
3.1	Calibrate a pendulum model	5
3.2	Rewrite a repeated identity with wildcards	6
3.3	Integrate a polynomial and verify it	7
3.4	Solve a nonlinear system exactly and numerically	7
3.5	Recover an interpolating polynomial with matrices	8
3.6	Differentiate once, evaluate a grid in one call	9
4	Result contracts	10
5	Limitations and troubleshooting	11
5.1	Capability boundaries	11
5.2	Common failures	11
6	API at a glance	11
7	API reference	12
7.1	Engine	12
7.2	Parsing, symbols, and rendering	12
7.3	Algebra and calculus	17
7.4	Rewriting	20
7.5	Evaluation and solving	25
7.6	Matrices	31
7.7	Scalar constructors	38
8	Compatibility, attribution, and licensing	41

1 Start here

Tymbolica is a focused Typst interface to a subset of [Symbolica](#). It supports exact expression construction, expansion and factorization, differentiation, polynomial integration, series, structural rewriting, exact and numerical system solving, batched evaluation, and rational-polynomial matrices. The algebra runs in a bundled WebAssembly plugin; the surrounding API and all rendering stay in Typst.

The shortest useful workflow is: parse an expression, transform it, then render the opaque result with `to-typst`.

```
#import "@local/tymbolica:0.1.0": *

#let p = math($(x + y)^3 - (x^3 + y^3)$)
#to-typst(factor(expand(p)))
```

$$3xy(x + y)$$

The result is exact: the expanded polynomial is collected and factored as $3xy(x + y)$ without converting its coefficients to floating point.

1.1 Installation

The repository currently provides a local package. On Linux, place or symlink the repository root at:

```
~/.local/share/typst/packages/local/tymbolica/0.1.0
```

Use the corresponding Typst data directory on macOS or Windows. The package root must contain `typst.toml`; its `typst` directory contains `lib.typ` and `tymbolica.wasm`. Then import:

```
#import "@local/tymbolica:0.1.0": *
```

From a source checkout, a document can instead import the library by relative path:

```
#import "path/to/tymbolica/typst/lib.typ": *
```

The checked-in WebAssembly bundle is ready to use. Rebuild it after changing the Rust bridge with `nix run .#build`. Build this manual with `nix run .#manual`, and compile every checked-in example with `nix run .#check`.

Package status The `@local` namespace is intentional for version 0.1.0. Replace it with the published namespace if Tymbolica is released through Typst Universe.

1.2 Choosing an operation

Goal	Primary functions	Result
Read Typst mathematics	<code>math</code> , <code>var</code> , <code>atom</code>	Opaque expression payload
Display or export	<code>to-typst</code> , <code>canonical</code> , <code>to-latex</code>	Typst content or source text
Exact algebra and calculus	<code>expand</code> , <code>factor</code> , <code>derivative</code> , <code>integrate</code> , <code>series</code>	Opaque expression payload
Structural rewriting	<code>wild</code> , <code>rule</code> , <code>replace</code>	Transformed expression payload
Evaluate many points	<code>evaluate-many</code> , <code>evaluate-grid</code>	Real/complex records and arrays
Solve equations	<code>solve-linear</code> , <code>solve-system</code> , <code>nsolve-system</code>	Exact solution rows or one numerical row
Exact linear algebra	<code>matrix</code> , <code>matrix-solve</code> , <code>row-reduce</code>	Matrix payloads or structured records

2 Core concepts

2.1 Expressions are opaque payloads

`math`, `var`, arithmetic constructors, transformations, solvers, and matrix operations return bytes. Those bytes are an internal Symbolica payload, not Typst math content. Keep them opaque and pass them to other Tymbolica functions. Render only at the document boundary:

Typst math $\rightarrow$ math $\rightarrow$ bytes $\rightarrow$ transform $\rightarrow$ bytes $\rightarrow$ to-typst $\rightarrow$ content

```
#let expression = math$(x + 1)^4$
#let transformed = expand(expression)

payload type: #type(transformed)\
document output: #to-typst(transformed)\
Symbolica text: #raw(canonical(transformed))
```

```
payload type: bytes
document output: 1 + 4x + 6x2 + 4x3 + x4
Symbolica text: 1+4*x+6*x^2+4*x^3+x^4
```

Use `to-typst` for normal document output. `to-typst-source` returns Typst math source, `to-latex` returns LaTeX source, and `canonical` returns Symbolica's canonical text representation. `canonical(namespaces: true)` is useful for debugging symbol identity; it is not a documented import format.

2.2 math, atom, var, and wild

Function	Typical input	Use
<code>math</code>	<code>\$x^2 + 1\$</code>	Parse Typst math through the bundled Parsely grammar.
<code>atom</code>	number, string, math, or bytes	Convert a general value to an expression payload.
<code>var</code>	<code>"x"</code>	Create an ordinary symbolic variable.
<code>wild</code>	<code>"a"</code>	Create a pattern placeholder such as <code>a_</code> .

var versus wild A variable is part of the mathematics. A wildcard exists only to capture a subexpression while matching a replacement pattern. If the expression is $x^2 + 1$, construct x with `var("x")`. Use `wild("a")` when a rule needs to match an arbitrary expression as `a_`.

Repeated occurrences of the same wildcard in one pattern must capture the same expression. Different matches can bind it differently; the rewriting guide below turns that rule into a concrete example.

2.3 Default functions and custom engines

For ordinary work, import and call the top-level functions. They share one default engine whose namespace is `"typst"`.

Use `init` only when you need a different default namespace, a custom parsing grammar, or a different plugin source. Built-in analytic functions such as `sin`, `cos`, and `exp` need Symbolica's namespace for calculus and numerical evaluation:

```
#let sym = init(namespace: "symbolica")
#let parse = sym.math
#let x = sym.var("x")
#let derivative = sym.derivative

#sym.to-typst(derivative(parse($sin(x)$), x))
```

Dictionary function fields must be bound first or called with parentheses: `(sym.derivative)(...)`. The examples bind short local names to keep the mathematics readable.

Symbols from different namespaces are distinct even when they print alike:

```
#let physics = init(namespace: "physics")
#let mixed = add(var("x"), (physics.var)("x"))

#raw(canonical(mixed, namespaces: true))
```

```
typst::x+physics::x
```

2.4 Systems and return ordering

Solver inputs are expressions understood to equal zero. For example, `math($x + y - 3$)` represents $x + y = 3$.

Exact system solvers return arrays of expression payloads. Columns follow the variable order supplied by the caller; the order of solution rows is not a stable contract. Numerical solvers return one floating-point solution near the initial guess.

3 Worked mathematical guides

Each guide asks a mathematical question, shows complete copyable code, renders the symbolic result, and performs a check. The examples are evaluated by Tidy while this manual compiles.

3.1 Calibrate a pendulum model

This workflow is adapted to Typst from Symbolica's [pendulum calibration example](#). Start from the potential $V(\theta) = \kappa(1 - \cos \theta)$, derive the restoring torque, approximate it for small angles, and infer the unknown scale κ and sensor offset τ_0 from two measurements.

```
#let sym = init(namespace: "symbolica")
#let m = sym.math
#let v = sym.var
#let render = sym.to-typst
#let derivative = sym.derivative
#let series = sym.series
#let replace = sym.replace
#let solve-linear = sym.solve-linear
#let evaluate-many = sym.evaluate-many
#let neg = sym.neg
#let add = sym.add
#let sub = sym.sub

// Parse before binding q and k, so Typst's math names stay content.
#let potential = m($kappa (1 - cos(theta))$)
#let q = v("θ")
#let k = v("κ")
#let b = v("τ₀")
#let q1 = v("θ₁")
#let q2 = v("θ₂")
#let t1 = v("τ₁")
#let t2 = v("τ₂")

#let torque = neg(derivative(potential, q))
#let small-angle = series(torque, q, 0, 3)
#let model = add(small-angle, b)

#let fit = solve-linear((
  sub(replace(model, q, q1), t1),
  sub(replace(model, q, q2), t2),
), (k, b))

#let fitted = evaluate-many(
  fit,
  (q1, q2, t1, t2),
  ((0.10, 0.20, -0.4697, -0.9545),),
).first()
#let predictions = evaluate-many(
  model,
  (q, k, b),
  (
    (0.10, fitted.at(0).re, fitted.at(1).re),
    (0.20, fitted.at(0).re, fitted.at(1).re),
  ),
)

$ V(theta) = #render(potential) $ \
$ tau(theta) = -(partial V)/(partial theta) = #render(torque) $ \
$ tau_small(theta) = #render(small-angle) $ \
$ kappa = #render(fit.at(0)) $ \
$ kappa approx #calc.round(fitted.at(0).re, digits: 6), quad
```

```
tau_0 approx #calc.round(fitted.at(1).re, digits: 6) $\
predicted torques:
#predictions.map(row => str(calc.round(row.first().re, digits: 4))).join(", ")
```

$$V(\theta) = \kappa(1 - \cos(\theta))$$

$$\tau(\theta) = -\frac{\partial V}{\partial \theta} = -\kappa \sin(\theta)$$

$$\tau_{\text{small}(\theta)} = -\kappa\theta + \frac{1}{6}\kappa\theta^3$$

$$\kappa = \frac{6 \tau_1 - 6 \tau_2}{-6 \theta_1 + 6 \theta_2 + \theta_1^3 - \theta_2^3}$$

$$\kappa \approx 4.905228, \quad \tau_0 \approx 0.020005$$

predicted torques: -0.4697, -0.9545

The symbolic result for κ is retained exactly until the final batched evaluation. The two predicted torques reproduce the measurements, which checks the substitution and solve rather than merely displaying the fitted formula.

Why the namespace matters In the default "typst" namespace, `cos` is an ordinary user function. The "symbolica" engine identifies it as Symbolica's analytic cosine, so differentiation, series expansion, and evaluation know its semantics.

3.2 Rewrite a repeated identity with wildcards

Suppose a long expression contains several copies of $\sin^2(a) + \cos^2(a)$. One wildcard must capture the same argument in both functions. Repeating the rule then handles each independently matched argument.

```
#let source = math(
  $3 (sin(x)^2 + cos(x)^2)
  + sin(x + y)^2 + cos(x + y)^2$
)
#let identity = math(
  $sin("a_")^2 + cos("a_")^2$
)

#let one-pass = replace(source, identity, 1)
#let reduced = replace(source, identity, 1, repeat: true)

source: #to-tyrst(source)\
one pass: #to-tyrst(one-pass)\
repeat to a fixed point: #to-tyrst(reduced)
```

source: $3(\sin(x)^2 + \cos(x)^2) + \sin(x + y)^2 + \cos(x + y)^2$
 one pass: $1 + 3(\sin(x)^2 + \cos(x)^2)$
 repeat to a fixed point: 4

Within one match, both occurrences of `a_` capture the same expression. Across matches it first captures one argument and then another. This is the practical difference between an ordinary `var("a")` and `wild("a")`.

Repeating rules `repeat: true` stops at a fixed point. Do not repeat mutually reversing or indefinitely growing rules: the current bridge has no public iteration limit for replacement loops.

3.3 Integrate a polynomial and verify it

Tymbolica currently integrates polynomials exactly. For a factored degree-five input, request the additive contributions, assemble the antiderivative, and differentiate the result to prove it.

```
#let x = var("x")
#let p = math((x + 1)^3 (2 x^2 - 3 x + 5))
#let integration = integrate-with-steps(p, x)
#let residual = expand(
  sub(derivative(integration.result, x), p)
)

$ p(x) = #to-tyrst(p) $
term contributions:
#for (index, term) in integration.steps.enumerate() [
  #(index + 1). #to-tyrst(term) #linebreak()
]
$ integral p(x) dif x = #to-tyrst(integration.result) + C $
verification: $ (partial I)/(partial x) - p(x) = #to-tyrst(residual) $
```

$$p(x) = (1 + x)^3(5 - 3x + 2x^2)$$

term contributions:

1. $5x$
2. $6x^2$
3. $\frac{8}{3}x^3$
4. $\frac{1}{2}x^4$
5. $\frac{3}{5}x^5$
6. $\frac{1}{3}x^6$

$$\int p(x) dx = 5x + 6x^2 + \frac{8}{3}x^3 + \frac{1}{2}x^4 + \frac{3}{5}x^5 + \frac{1}{3}x^6 + C$$

verification:

$$\frac{\partial I}{\partial x} - p(x) = 0$$

`integrate-with-steps` returns (`result: bytes`, `steps: array`). Here `steps` are the additive antiderivative contributions produced after polynomial expansion; they are not Symbolica's full pedagogical integration tree. The payload represents one antiderivative, so the manual adds the conventional arbitrary constant C when displaying an indefinite integral.

Integration boundary Denominators depending on the integration variable are currently rejected. In particular, Symbolica's richer examples such as $\int \frac{1}{1+x^2} dx$ are outside Tymbolica's present bridge.

3.4 Solve a nonlinear system exactly and numerically

Intersect the circle $x^2 + y^2 = 25$ with the line $x - y = 1$. Exact solving should return every algebraic branch. Numerical solving should select a branch near the supplied seed.

```
#let x = var("x")
#let y = var("y")
#let system = (
  math($x^2 + y^2 - 25$),
  math($x - y - 1$),
)

#let exact = solve-system(system, (x, y))
#let positive = nsolve-system(
  system, (x, y), (3.0, 3.0), prec: 1e-10,
)
#let negative = nsolve-system(
  system, (x, y), (-3.0, -3.0), prec: 1e-10,
)
#let checks = evaluate-many(
  system, (x, y), (positive, negative),
)
#let max-residual(row) = row.map(
```

```

value => calc.abs(value.re),
).sorted().last()

exact branches:\
#for (index, row) in exact.enumerate() [
  #(index + 1). $x = #to-tyrst(row.at(0)), y = #to-tyrst(row.at(1))$
  #linebreak()
]
seed $(3,3)$ $arrow.r$
(#positive.map(value => str(calc.round(value, digits: 6))).join(", "));
maximum residual #repr(max-residual(checks.first()))\
seed $(-3,-3)$ $arrow.r$
(#negative.map(value => str(calc.round(value, digits: 6))).join(", "));
maximum residual #repr(max-residual(checks.last()))

```

exact branches:

```

1. x = 4, y = 3
2. x = -3, y = -4
seed (3,3) → (4, 3); maximum residual 0.0
seed (-3,-3) → (-3, -4); maximum residual 3.552713678800501e-15

```

Every input expression means “equal to zero.” Each exact row is ordered as (x, y) because that is the requested variable order. Numerical results use f64; small nonzero residuals near machine precision are expected. A different seed may converge to another root or fail to converge.

3.5 Recover an interpolating polynomial with matrices

Find the quadratic through (0, 1), (1, 3), and (2, 8). Writing $p(t) = a_0 + a_1t + a_2t^2$ turns interpolation into the exact matrix equation $Aa = b$.

```

#let t = var("t")
#let A = matrix((
  (1, 0, 0),
  (1, 1, 1),
  (1, 2, 4),
))
#let b = vec((1, 3, 8))
#let coefficients = matrix-solve(A, b)
#let check = matrix-mul(A, coefficients)
#let reduced = row-reduce(
  augment(A, b),
  max-col: 3,
)
#let polynomial = add(
  matrix-at(coefficients, 0, 0),
  mul(matrix-at(coefficients, 1, 0), t),
  mul(matrix-at(coefficients, 2, 0), pow(t, 2)),
)

$ A = #to-tyrst(A), quad b = #to-tyrst(b) $\
$ op("det")(A) = #to-tyrst(det(A)) $\
$ a = #to-tyrst(coefficients) $\
$ p(t) = #to-tyrst(polynomial) $\
$ A a = #to-tyrst(check) $\
rank: #reduced.rank; augmented RREF: #to-tyrst(reduced.matrix)

```


$$A = \begin{pmatrix} 1 & 0 & 0 \\ 1 & 1 & 1 \\ 1 & 2 & 4 \end{pmatrix}, \quad b = \begin{pmatrix} 1 \\ 3 \\ 8 \end{pmatrix}$$

$$\det(A) = 2$$

$$a = \begin{pmatrix} 1 \\ \frac{1}{2} \\ \frac{3}{2} \end{pmatrix}$$

$$p(t) = 1 + \frac{1}{2}t + \frac{3}{2}t^2$$

$$Aa = \begin{pmatrix} 1 \\ 3 \\ 8 \end{pmatrix}$$

$$\text{rank: 3 augmented RREF: } \begin{pmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & \frac{1}{2} \\ 0 & 0 & 1 & \frac{3}{2} \end{pmatrix}$$

`vec` constructs a column matrix, and matrix indices are zero-based. `max-col: 3` tells row-reduce to determine pivots and rank from the three coefficient columns rather than from the augmented right-hand side. The multiply-back check recovers the original measurement vector.

3.6 Differentiate once, evaluate a grid in one call

For $f(x, y) = x^2 + xy + y^2$, derive both gradient components symbolically and then evaluate all three expressions over a Cartesian grid. This is both clearer and cheaper than issuing one WebAssembly call per cell and per expression.

```
#let x = var("x")
#let y = var("y")
#let f = math($x^2 + x y + y^2$)
#let fx = derivative(f, x)
#let fy = derivative(f, y)
#let grid = evaluate-grid(
  (f, fx, fy),
  (x, y),
  (
    domain(-1, 1, samples: 3),
    domain(-1, 1, samples: 3),
  ),
)
#let cells = range(grid.points.len()).map(index => {
  let point = grid.points.at(index)
  let values = grid.values.at(index)
  (
    [#point.at(0)],
    [#point.at(1)],
    [#values.at(0).re],
    [#values.at(1).re],
    [#values.at(2).re],
  )
}).flatten()

$ f = #to-tyrst(f) $
$ partial_x f = #to-tyrst(fx), quad partial_y f = #to-tyrst(fy) $

#table(
  columns: 5,
  inset: 4pt,
  stroke: 0.35pt + rgb("#d5dbe1"),
  table.header([$x$], [$y$], [$f$], [$partial_x f$], [$partial_y f$]),
  ..cells,
)
```

$$f = xy + x^2 + y^2$$

$$\partial_x f = 2x + y, \quad \partial_y f = x + 2y$$

x	y	f	$\partial_x f$	$\partial_y f$
-1	-1	3	-3	-3
-1	0	1	-2	-1
-1	1	1	-1	1
0	-1	1	-1	-2
0	0	0	0	0
0	1	1	1	2
1	-1	1	1	-1
1	0	1	2	1
1	1	3	3	3

The returned dictionary has three fields:

Field	Type	Meaning
shape	array	Sample count along each variable axis.
points	array	Flattened real coordinate rows.
values	array	Rows of (re, im) results, one per expression.

The last domain varies fastest in the flattened arrays. Use `evaluate-many` instead when the points are irregular rather than a Cartesian product.

4 Result contracts

Most symbolic functions return opaque expression or matrix bytes. Functions that need metadata return Typst dictionaries or arrays:

Function	Shape	Contract
<code>evaluate</code>	dictionary	(re: float, im: float)
<code>integrate-with-steps</code>	dictionary	result: bytes; steps: array of additive contributions
<code>evaluate-many</code>	array	One result row per point; one complex value per expression
<code>evaluate-grid</code>	dictionary	shape, flattened points, flattened values
<code>solve-linear</code>	array	One exact expression per requested variable
<code>solve-system</code>	array	One row per exact solution; columns follow variable order
<code>nsolve-system</code>	array	One floating-point value per requested variable
<code>row-reduce</code>	dictionary	matrix: bytes; rank: int
<code>split-col</code>	array	Left and right matrix payloads

Real evaluation results still use the complex record with `im: 0.0`. That uniform shape lets the same code handle expressions that later become complex.

5 Limitations and troubleshooting

5.1 Capability boundaries

- Tymbolica exposes a selected Symbolica surface, not Symbolica's complete Python or Rust API. Polynomial objects, Gröbner bases, `collect`, `together`, `apart`, `cancel`, persistent/JIT evaluators, arbitrary precision, and the full integration package are not currently exposed.
- Exact integration currently accepts polynomial expressions only and returns no integration constant in its payload. Its additive contributions follow Symbolica's canonical expanded-term order rather than the source-text order.
- `solve-system` targets linear or polynomial nonlinear systems. `nsolve` and `nsolve-system` use `f64`, depend on an initial guess, and return no iteration or convergence metadata beyond success or failure.
- Matrix entries must be compatible with Symbolica rational-polynomial coefficients. Matrices must be nonempty and dimensionally compatible.
- Structural transformations return expressions, not assumption or domain records. If a manipulation is valid only under a condition such as $x \neq 1$, preserve that condition in the surrounding document.
- The parser covers the Typst math structures represented by the bundled Parsely grammar. Use `array-tree` to inspect unfamiliar input.

5.2 Common failures

Symptom	Likely cause	What to try
expected content, found bytes	An opaque Tymbolica payload was inserted directly into <code>\$. . . \$</code> .	Render it with <code>to-typst</code> , or keep it outside Typst math construction.
A derivative or series leaves a function unchanged	The function lives in the default user namespace.	Use <code>init(namespace: "symbolica")</code> for Symbolica built-ins.
A replacement does not match	Pattern and source use different namespaces, or a repeated wildcard would capture unequal expressions.	Inspect <code>canonical(..., namespaces: true)</code> and simplify the pattern.
An exact solver errors	The system is inconsistent or outside the supported polynomial surface.	Confirm that every item means <code>= 0</code> ; try <code>nsolve-system</code> for a numerical branch.
A numerical solver errors or finds an unwanted root	The seed is poor or selects a different basin.	Try another physically meaningful initial guess and check residuals.
A matrix operation errors	Shapes differ, a matrix is singular, or an entry is unsupported.	Inspect <code>matrix-shape</code> , <code>det</code> , and the coefficient expressions.

6 API at a glance

Area	Functions
Engine	<code>init</code>
Parsing and rendering	<code>math</code> , <code>atom</code> , <code>var</code> , <code>wild</code> , <code>array-tree</code> , <code>canonical</code> , <code>to-typst-source</code> , <code>to-typst</code> , <code>to-latex</code>
Algebra and calculus	<code>simplify</code> , <code>expand</code> , <code>factor</code> , <code>derivative</code> , <code>integrate</code> , <code>integrate-with-steps</code> , <code>series</code>
Rewriting	<code>rule</code> , <code>replace</code> , <code>replace-multiple</code> , <code>replace-wildcards</code>
Evaluation and solving	<code>evaluate</code> , <code>domain</code> , <code>evaluate-many</code> , <code>evaluate-grid</code> , <code>solve-linear</code> , <code>solve-system</code> , <code>nsolve</code> , <code>nsolve-system</code>

Area	Functions
Matrices	matrix, vec, identity, eye, matrix-add, matrix-sub, matrix-mul, matrix-div-scalar, transpose, det, inv, matrix-solve, matrix-solve-any, row-reduce, augment, split-col, primitive-part, content, matrix-at, matrix-shape
Scalar constructors	add, mul, neg, sub, div, pow

7 API reference

The reference is generated from the doc comments in `lib.typ` with Tidy. Its examples import the real top-level exports, so compiling this manual also exercises those wrappers.

7.1 Engine

7.1.1 init

Create an independent Symbolica engine backed by a WebAssembly plugin.

The returned dictionary exposes the same parsing, algebra, evaluation, solving, and matrix functions as this module's top-level API. Its methods use this engine's namespace, source, and grammar; the top-level functions use one shared engine initialized with the defaults below. Create an engine when you need a different symbol namespace, plugin location, or grammar.

```
#let sym = init(namespace: "physics")
#let v = sym.var
#let render = sym.canonical
#raw(render(v("x"), namespaces: true))
```

```
physics::x
```

Parameters

```
init(
  namespace: str,
  source: str,
  grammar: dictionary
) -> dictionary
```

namespace str

Default namespace for symbols parsed from Typst math or strings. A per-call namespace passed to `math` or `var` takes precedence.

Default: `"typst"`

source str

WebAssembly plugin path passed to Typst's plugin constructor. Relative paths are resolved by Typst from this source file.

Default: `"tymbolica.wasm"`

grammar dictionary

Parsely grammar used by `math` and `array-tree` unless they receive an explicit grammar override.

Default: `_default_grammar`

7.2 Parsing, symbols, and rendering

7.2.1 math

Parse Typst math content into an opaque Symbolica atom payload.

Arithmetic, fractions, powers, roots, absolute values, calls, and common Typst math structures are translated through Parsely. Matrix-valued `mat(...)` and `vec(...)` content must instead be passed to `matrix` or `vec`. Keep the returned bytes opaque and use this module's functions to inspect or transform them.

```
#let expr = math($x + 1$)
#to-typst(expr)
```

 $1 + x$

Parameters

```
math(
  eqn: content,
  grammar: dictionary | none,
  namespace: str | none
) -> bytes
```

eqn `content`

Math content, normally written as `$. . . $`.

grammar `dictionary` or `none`

Parsely grammar override. `none` uses the default engine grammar.

Default: `none`

namespace `str` or `none`

Namespace for parsed symbols. `none` uses the engine namespace ("typst" for the top-level function).

Default: `none`

7.2.2 atom

Convert a supported Typst value or math expression into an atom payload.

Existing atom bytes pass through unchanged. Content is parsed like `math`; integers and floats become numbers. Strings are parsed as leaf values: numeric strings become numbers and other strings become symbols in the engine's namespace. Matrix payloads are not atom payloads and should not be passed to atom-only algebra functions.

```
#to-typst(atom("x"))
```

 x

Parameters

```
atom(value: bytes | content | int | float | str) -> bytes
```

value `bytes` or `content` or `int` or `float` or `str`

Value to convert.

7.2.3 var

Construct a named Symbolica variable.

Unlike `wild`, this is an ordinary mathematical symbol and therefore does not capture subexpressions during pattern matching.

```
#to-tpst(var("x"))
```

 x

Parameters

```
var(
  name: str,
  namespace: str none
) -> bytes
```

name `str`

Symbol name without a namespace prefix or wildcard suffix.

namespace `str` or `none`

Namespace override. `none` uses the engine namespace.

Default: `none`

7.2.4 wild

Construct a Symbolica pattern wildcard.

This creates the symbol name followed by `level` underscores. A wildcard is a pattern placeholder used by `rule`, `replace`, and `replace-wildcards`; it is not an ordinary unknown for algebra or solving. Use `var` for a mathematical variable.

```
#raw(canonical(wild("a")))
```

 $a_$

Parameters

```
wild(
  name: str,
  level: int,
  namespace: str none
) -> bytes
```

name `str`

Base name of the wildcard, without trailing underscores.

level `int`

Number of underscore levels to append. 1 creates a conventional single wildcard such as `a_`; 0 appends no underscore and therefore creates an ordinary symbol instead.

Default: `1`

namespace `str` or `none`

Namespace override. `none` uses the engine namespace.

Default: **none**

7.2.5 array-tree

Render the Parsely parse tree for a Typst math expression.

This is a diagnostic view of the tree consumed by math; it does not create a Symbolica atom.

```
#array-tree($(x + 1)^2$)
```

```
(
  strong(body: raw(text: "pow")),
  (
    styled(child: raw(text: "base"), ..),
    (
      strong(body: raw(text: "()")),
      (
        styled(child: raw(text: "expr"), ..),
        (
          strong(body: raw(text: "add")),
          equation(body: [x]),
          equation(body: [1]),
        ),
      ),
    ),
  ),
  (
    styled(child: raw(text: "exp"), ..),
    equation(body: [2]),
  ),
)
```

Parameters

```
array-tree(
  eqn: content,
  grammar: dictionary or none
) -> content
```

eqn **content**

Math content to inspect.

grammar **dictionary** or **none**

Parsely grammar override. none uses the engine grammar.

Default: **none**

7.2.6 canonical

Render an atom or matrix payload as Symbolica source text.

```
#raw(canonical(math($x + 1$)))
```

```
1+x
```

Parameters

```
canonical(
  expr: bytes content int float str,
  namespaces: bool
) -> str
```

expr bytes or content or int or float or str

Atom or matrix payload. Other supported expression values are first converted as by atom.

namespaces bool

Include symbol namespaces in the output.

Default: **false**

7.2.7 to-typst-source

Render an atom or matrix payload as Typst math source.

```
#raw(to-typst-source(math($x + 1$)))
```

```
1+x
```

Parameters

```
to-typst-source(expr: bytes content int float str) -> str
```

expr bytes or content or int or float or str

Atom or matrix payload, or a supported expression value.

7.2.8 to-typst

Render an atom or matrix payload as evaluated Typst math content.

The payload is first printed as Typst math source and then evaluated in math mode. Use `to-typst-source` when you need the source string instead.

```
#to-typst(math($x + 1$))
```

```
1 + x
```

Parameters

```
to-typst(
  expr: bytes content int float str,
  block: bool
) -> content
```

expr bytes or content or int or float or str

Atom or matrix payload, or a supported expression value.

block bool

Render as a block equation instead of inline math.

Default: `false`

7.2.9 to-latex

Render an atom or matrix payload as LaTeX source.

```
#raw(to-latex(math($x^2 + 1$)))
```

 $1+x^2$

Parameters

`to-latex`(`expr`: `bytes` `content` `int` `float` `str`) -> `str`

expr `bytes` or `content` or `int` or `float` or `str`

Atom or matrix payload, or a supported expression value.

7.3 Algebra and calculus

7.3.1 simplify

Re-import and re-export an atom in Symbolica's canonical internal form.

This operation does not perform algebraic simplification; it currently acts as a normalization and validation round-trip for atom payloads.

```
#to-tyrst(simplify(math($x + 0$)))
```

 x

Parameters

`simplify`(`expr`: `bytes` `content` `int` `float` `str`) -> `bytes`

expr `bytes` or `content` or `int` or `float` or `str`

Atom payload or supported expression value to normalize.

7.3.2 expand

Expand an expression through Symbolica's polynomial expansion.

```
#to-tyrst(expand(math($ (x + 1)^2$)))
```

 $1 + 2x + x^2$

Parameters

`expand`(`expr`: `bytes` `content` `int` `float` `str`) -> `bytes`

expr `bytes` or `content` or `int` or `float` or `str`

Atom payload or supported expression value to expand.

7.3.3 factor

Factor an expression with Symbolica's exact factorization routine.

```
#to-tyrst(factor(math($x^2 + 2 x + 1$)))
```

$$(1 + x)^2$$

Parameters

```
factor(expr: bytes content int float str) -> bytes
```

expr bytes or content or int or float or str

Atom payload or supported expression value to factor.

7.3.4 derivative

Differentiate an expression exactly with respect to an indeterminate.

```
#let x = var("x")
#to-tyrst(derivative(math($(x + 1)^2$), x))
```

$$2(1 + x)$$

Parameters

```
derivative(
  expr: bytes content int float str,
  var: bytes content str
) -> bytes
```

expr bytes or content or int or float or str

Expression to differentiate.

var bytes or content or str

Symbolica indeterminate, normally created with var.

7.3.5 integrate

Integrate a polynomial exactly with respect to a variable.

The expression must be polynomial in var; rational functions with a non-constant denominator and general transcendental integrands are rejected. No arbitrary integration constant is added.

```
#let x = var("x")
#to-tyrst(integrate(math($3 x^2 - 4 x + 7$),
x))
```

$$7x - 2x^2 + x^3$$

Parameters

```
integrate(
  expr: bytes content int float str,
  var: bytes content str
) -> bytes
```

expr bytes or content or int or float or str

Polynomial expression to integrate.

var bytes or content or str

Integration variable, normally created with var.

7.3.6 integrate-with-steps

Integrate a polynomial and expose one contribution per expanded term.

This has the same polynomial-only restriction as `integrate`. It returns `(result: bytes, steps: array)`, where `result` is the complete antiderivative and every atom payload in `steps` is the antiderivative of one top-level term after Symbolica expands and canonically normalizes the input. A non-sum has one step; source terms may combine or cancel during expansion. Thus `steps` is a deterministic term-by-term decomposition, not a general pedagogical derivation trace. No integration constant is added.

```
#let x = var("x")
#let out = integrate-with-steps(math($3 x^2 - 4
x + 7$), x)
#to-tyrst(out.result)
#out.steps.map(to-tyrst).join[, ]
```

$$7x - 2x^2 + x^3 \quad 7x, \quad -2x^2, \quad x^3$$

Parameters

```
integrate-with-steps(
  expr: bytes content int float str,
  var: bytes content str
) -> dictionary
```

expr bytes or content or int or float or str

Polynomial expression to integrate.

var bytes or content or str

Integration variable, normally created with var.

7.3.7 series

Compute a univariate series expansion around expansion-point.

The truncation depth is the rational number `depth / depth-denom`. With an absolute depth it is measured directly in var; with a relative depth it is measured from the lowest order encountered in the expression.

```
#let sym = init(namespace: "symbolica")
#let m = sym.math
#let v = sym.var
#let ser = sym.series
#let render = sym.to-tyrst
#render(ser(m($cos(x)/(x + 1$)), v("x"), 0, 3))
```

$$1 - x + \frac{1}{2}x^2 - \frac{1}{2}x^3$$

Parameters

```
series(
  expr: bytes content int float str,
  var: bytes content str,
  expansion-point: bytes content int float str,
  depth: int,
  depth-denom: int,
  depth-is-absolute: bool
) -> bytes
```

expr bytes or content or int or float or str

Expression to expand.

var bytes or content or str

Expansion variable, normally created with var.

expansion-point bytes or content or int or float or str

Point about which to expand.

depth int

Numerator of the requested truncation depth.

depth-denom int

Denominator of the truncation depth.

Default: 1

depth-is-absolute bool

Use an absolute depth when true, or a depth relative to the lowest order in the expression when false.

Default: true

7.4 Rewriting

7.4.1 rule

Build a reusable replacement rule for replace-multiple.

pattern and rhs may contain symbols created with wild. The returned dictionary contains opaque atom payloads plus the matching options; pass it to replace-multiple rather than editing it manually.

```
#let r = rule(math($f("a_")$), math($g("a_")$))
#to-tyrst(replace-multiple(math($f(x)$), (r,)))
```

$g(x)$

Parameters

```
rule(
  pattern: bytes content int float str,
  rhs: bytes content int float str,
  non-greedy-wildcards: array,
  min-level: int,
  max-level: int none,
```

```

    level-range: array none,
    level-is-tree-depth: bool,
    partial: bool,
    allow-new-wildcards-on-rhs: bool,
    rhs-cache-size: int
) -> dictionary

```

pattern bytes or content or int or float or str

Pattern to match.

rhs bytes or content or int or float or str

Expression substituted for each match. Wildcards captured by pattern are substituted in this expression.

non-greedy-wildcards array

Wildcards that should prefer the smallest possible match.

Default: ()

min-level int

Lowest expression level at which matching is allowed; the root is level zero.

Default: 0

max-level int or none

Highest allowed matching level, or none for no upper bound.

Default: none

level-range array or none

Optional (minimum, maximum) pair overriding min-level and max-level; the maximum may be none.

Default: none

level-is-tree-depth bool

Count full expression-tree depth when true; otherwise levels increase when entering functions.

Default: false

partial bool

Allow a pattern to match part of a sum, product, or other term instead of requiring the whole term.

Default: true

allow-new-wildcards-on-rhs bool

Permit wildcards on rhs that do not occur in pattern. When false, such rules are rejected.

Default: `false`

rhs-cache-size `int`

Maximum number of substituted right-hand sides cached; use zero to disable this cache.

Default: `100`

7.4.2 replace

Replace subexpressions matching pattern with rhs.

By default all non-overlapping outermost matches are replaced once. `repeat: true` reapplies the rule until the expression stops changing; rules that cycle therefore do not terminate. Use rule plus `replace-multiple` when several patterns must be considered together.

```
#to-typst(replace(math($f(x, y)$),
math($f("a_", "b_")$), math($g("b_", "a_")$)))
```

$$g(y, x)$$

Parameters

```
replace(
  expr: bytes content int float str,
  pattern: bytes content int float str,
  rhs: bytes content int float str,
  repeat: bool,
  once: bool,
  bottom-up: bool,
  nested: bool,
  non-greedy-wildcards: array,
  min-level: int,
  max-level: int none,
  level-range: array none,
  level-is-tree-depth: bool,
  partial: bool,
  allow-new-wildcards-on-rhs: bool,
  rhs-cache-size: int
) -> bytes
```

expr bytes or content or int or float or str

Expression in which to replace matches.

pattern bytes or content or int or float or str

Pattern to match.

rhs bytes or content or int or float or str

Replacement expression.

repeat bool

Reapply the rule until it makes no further change.

Default: `false`

once `bool`

Replace only the first match found during a pass.

Default: `false`

bottom-up `bool`

Visit deepest matches before outer matches.

Default: `false`

nested `bool`

Replace nested matches from the deepest outward, acting on the result of each inner replacement.

Default: `false`

non-greedy-wildcards `array`

Wildcards that should prefer the smallest possible match.

Default: `()`

min-level `int`

Lowest allowed matching level; the root is level zero.

Default: `0`

max-level `int` or `none`

Highest allowed matching level, or none for no upper bound.

Default: `none`

level-range `array` or `none`

Optional (minimum, maximum) pair overriding min-level and max-level.

Default: `none`

level-is-tree-depth `bool`

Count full tree depth instead of function-entry depth.

Default: `false`

partial `bool`

Allow matching a part of a term rather than the entire term.

Default: `true`

allow-new-wildcards-on-rhs `bool`

Permit rhs wildcards that are absent from pattern.

Default: `false`

rhs-cache-size `int`

Maximum number of substituted right-hand sides cached; zero disables the cache.

Default: `100`

7.4.3 replace-multiple

Apply several reusable replacement rules together.

The traversal options apply to the combined rule set. With `repeat: true`, the complete set is reapplied until no rule changes the expression; cyclic rule sets do not terminate.

```
#let r1 = rule(math($f("a_")$),
math($h("a_")$))
#let r2 = rule(var("x"), var("z"))
#to-tyrst(replace-multiple(math($f(x) + x$),
(r1, r2)))
```

$$z + h(x)$$

Parameters

```
replace-multiple(
  expr: bytes content int float str,
  rules: array,
  repeat: bool,
  once: bool,
  bottom-up: bool,
  nested: bool
) -> bytes
```

expr `bytes` or `content` or `int` or `float` or `str`

Expression in which to replace matches.

rules `array`

Array of dictionaries returned by rule.

repeat `bool`

Reapply the rule set until it makes no further change.

Default: `false`

once `bool`

Replace only the first match found during a pass.

Default: `false`

bottom-up `bool`

Visit deepest matches before outer matches.

Default: `false`

nested `bool`

Replace nested matches from deepest to outermost, acting on intermediate results.

Default: `false`

7.4.4 replace-wildcards

Substitute explicit values for wildcard placeholders in a pattern.

This does not search another expression. It transforms pattern itself and requires each key in replacements to be a wildcard symbol.

```
#to-tyrst(replace-wildcards(math($k("a_")$),
  ((wild("a"), math($x + 1$))),))
```

$$k(1 + x)$$

Parameters

```
replace-wildcards(
  pattern: bytes content int float str,
  replacements: array
) -> bytes
```

pattern bytes or content or int or float or str

Pattern containing wildcards to substitute.

replacements array

Array of (wildcard, replacement) pairs.

7.5 Evaluation and solving

7.5.1 evaluate

Evaluate one expression numerically with optional substitutions.

values maps atom keys to real numbers or complex dictionaries of the form (re: number, im: number). Evaluation must eliminate every unsupported symbolic quantity. The result always has the exact shape (re: float, im: float), even when it is real.

```
#let x = var("x")
#let y = var("y")
#repr(evaluate(math($x^2 + y$), values: ((x,
  2.0), (y, 3.0))))
```

$$(re: 7.0, im: 0.0)$$

Parameters

```

evaluate(
  expr: bytes content int float str,
  values: array
) -> dictionary

```

expr bytes or content or int or float or str

Expression to evaluate.

values array

Array of (expression, value) substitution pairs. Values may be real numbers or (re: ..., im: ...) dictionaries.

Default: ()

7.5.2 domain

Describe one real sampling axis for evaluate-grid.

Both endpoints are included when samples is greater than one. A single-sample domain evaluates only at min. The returned dictionary has the exact shape (min: float, max: float, samples: int).

```
#repr(domain(-1, 1, samples: 3))
```

```
(min: -1.0, max: 1.0, samples: 3)
```

Parameters

```

domain(
  min: int float,
  max: int float,
  samples: int
) -> dictionary

```

min int or float

Finite lower endpoint; it must be less than max.

max int or float

Finite upper endpoint.

samples int

Number of evenly spaced points. Must be positive.

Default: 200

7.5.3 evaluate-many

Evaluate one or more expressions at explicit points in one Wasm call.

A single expression or variable may be passed directly; otherwise use an array. Every point must supply one real or complex value per variable. The result has one row per point in input order and one (re: float, im: float) dictionary per expression in expression order.

```
#let x = var("x")
#let y = var("y")
#let rows = evaluate-many((math($x + y$),
math($x y$)), (x, y), ((1, 2), (3, 4)))
#repr(rows)
```

```
(
  ((re: 3.0, im: 0.0), (re: 2.0, im: 0.0)),
  ((re: 7.0, im: 0.0), (re: 12.0, im: 0.0)),
)
```

Parameters

```
evaluate-many(
  expressions: bytes content array int float str,
  variables: bytes content array str,
  points: array
) -> array
```

expressions bytes or content or array or int or float or str

One expression or a non-empty array of expressions to evaluate.

variables bytes or content or array or str

One variable or an array defining input-column order.

points array

Array of input rows. Each row is an array ordered like variables; for one variable, a scalar point is also accepted.

7.5.4 evaluate-grid

Evaluate expressions over a Cartesian product of real domains in one Wasm call.

The result is (shape: array, points: array, values: array). shape lists the sample count of every domain. The grid axes are flattened into rows with the last domain varying fastest: each points row contains real coordinates in variable order, while the corresponding values row contains one (re: float, im: float) dictionary per expression.

```
#let x = var("x")
#let y = var("y")
#let grid = evaluate-grid(math($x^2 + y$), (x,
y), (domain(-1, 1, samples: 3), domain(0, 1,
samples: 2)))
shape: #repr(grid.shape); values:
#repr(grid.values)
```

```
shape: (3, 2) values: (
  ((re: 1.0, im: 0.0),),
  ((re: 2.0, im: 0.0),),
  ((re: 0.0, im: 0.0),),
  ((re: 1.0, im: 0.0),),
  ((re: 1.0, im: 0.0),),
  ((re: 2.0, im: 0.0),),
)
```

Parameters

```
evaluate-grid(
  expressions: bytes content array int float str,
  variables: bytes content array str,
```

```
domains: array
) -> dictionary
```

expressions bytes or content or array or int or float or str

One expression or a non-empty array of expressions.

variables bytes or content or array or str

One variable or an array defining grid-axis order.

domains array

One domain dictionary per variable, in the same order.

7.5.5 solve-linear

Solve a linear system exactly for variables.

Each item in system is interpreted as an expression equal to zero. A vector matrix may be supplied instead of an array. The returned array contains one atom payload per item in variables, in the same order. For an underdetermined system, bound variables may be expressed using free variables; Symbolica chooses the highest-indexed variables as free. Inconsistent or nonlinear systems produce an error.

```
#let x = var("x")
#let y = var("y")
#let sol = solve-linear((math($2 x + y - 5$),
math($x - y - 1$)), (x, y))
#sol.map(to-tyrst).join[, ]
```

2, 1

Parameters

```
solve-linear(
  system: array bytes content int float str,
  variables: array content str
) -> array
```

system array or bytes or content or int or float or str

Linear expressions understood to equal zero, as an array or vector matrix.

variables array or content or str

Variables to solve for, in result order.

7.5.6 solve-system

Solve a linear or supported polynomial nonlinear system exactly.

Each expression in system is understood to equal zero. Polynomial systems are solved exactly through Symbolica's Gröbner-basis and algebraic-root machinery; coefficients may contain symbolic parameters when Symbolica can treat them rationally. The result is an array of solution rows. Every row contains one atom payload per requested variable, in variables order; an empty array means there are no solutions.

```
#let x = var("x")
#let y = var("y")
#let solutions = solve-system((math($x^2 - 1$),
math($y - x$)), (x, y))
#repr(solutions.map(row => row.map(canonical)))
```

```
(("-1", "-1"), ("1", "1"))
```

Parameters

```
solve-system(
  system: array content int float str,
  variables: array content str
) -> array
```

system array or content or int or float or str

Expressions understood to equal zero.

variables array or content or str

Variables to eliminate and return, in result-column order.

7.5.7 nsolve

Find a real root of a univariate expression with Newton's method.

The expression is interpreted as equal to zero and evaluated with f64 arithmetic. Convergence is local and depends on init; failure to converge within max-iterations produces an error.

```
#let x = var("x")
#repr(nsolve(math($x^2 - 2$), x, 1.0))
```

```
1.4142135623746899
```

Parameters

```
nsolve(
  expr: bytes content int float str,
  var: bytes content str,
  init: int float,
  prec: int float,
  max-iterations: int
) -> float
```

expr bytes or content or int or float or str

Expression understood to equal zero.

var bytes or content or str

Real solve variable, normally created with var.

init int or float

Initial real guess.

prec int or float

Numerical tolerance for the Newton iteration.

Default: `1e-4`

max-iterations `int`

Maximum number of Newton iterations.

Default: `1000`

7.5.8 nsolve-system

Find a common real root of a system with multivariate Newton iteration.

Every expression is interpreted as equal to zero. `variables` and `init` must have matching lengths, and the returned floats follow that same order. Convergence is local and is not guaranteed for an arbitrary initial guess.

```
#let x = var("x")
#let y = var("y")
#repr(nsolve-system((math($x^2 + y - 3$),
math($x - y$)), (x, y), (1.0, 1.0)))
```

```
(1.3027756377319948, 1.3027756377319948)
```

Parameters

```
nsolve-system(
  system: array content int float str,
  variables: array content str,
  init: array,
  prec: int float,
  max-iterations: int
) -> array
```

system `array` or `content` or `int` or `float` or `str`

Expressions understood to equal zero.

variables `array` or `content` or `str`

Variables to solve for, in result order.

init `array`

Initial real value for every variable.

prec `int` or `float`

Numerical tolerance for the Newton iteration.

Default: `1e-4`

max-iterations `int`

Maximum number of Newton iterations.

Default: 1000

7.6 Matrices

7.6.1 matrix

Convert Typst values into an opaque Symbolica matrix payload.

Accepted forms are Typst math `mat(...)` or `vec(...)`, a non-empty rectangular nested array, a flat array interpreted as a column vector, a scalar interpreted as a one-by-one matrix, or an existing payload. Every entry must be convertible to a rational polynomial; general transcendental expressions are not valid matrix entries.

```
#to-typst(matrix($mat(1, 2; 3, 4)$))
```

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$$

Parameters

`matrix(value: bytes | content | array | int | float | str) -> bytes`

value bytes or content or array or int or float or str

Matrix source value.

7.6.2 vec

Convert a non-empty sequence into a column-vector matrix payload.

Array entries must be rational-polynomial compatible. Typst math `vec(...)` is accepted directly; existing matrix bytes pass through unchanged.

```
#to-typst(vec((1, 2)))
```

$$\begin{pmatrix} 1 \\ 2 \end{pmatrix}$$

Parameters

`vec(values: array | content | bytes) -> bytes`

values array or content or bytes

Vector entries, Typst `vec(...)` content, or an existing matrix payload.

7.6.3 identity

Create an n by n identity matrix payload.

```
#to-typst(identity(2))
```

$$\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$

Parameters

`identity(n: int) -> bytes`

n int

Positive matrix dimension.

7.6.4 eye

Create a square diagonal matrix from a non-empty entry array.

Every entry must be rational-polynomial compatible.

```
#to-tyrst(eye((1, 2)))
```

$$\begin{pmatrix} 1 & 0 \\ 0 & 2 \end{pmatrix}$$

Parameters

`eye(diag: array) -> bytes`

diag array

Diagonal entries in top-left to bottom-right order.

7.6.5 matrix-add

Add two matrices entry by entry.

The matrices must have equal shapes. Matrix source values accepted by `matrix` may be supplied directly.

```
#to-tyrst(matrix-add(matrix(((1, 2), (3, 4))),
identity(2)))
```

$$\begin{pmatrix} 2 & 2 \\ 3 & 5 \end{pmatrix}$$

Parameters

```
matrix-add(
  lhs: bytes content array int float str,
  rhs: bytes content array int float str
) -> bytes
```

lhs bytes or content or array or int or float or str

Left matrix.

rhs bytes or content or array or int or float or str

Right matrix of the same shape.

7.6.6 matrix-sub

Subtract two matrices entry by entry.

The matrices must have equal shapes. Matrix source values accepted by `matrix` may be supplied directly.

```
#to-tyrst(matrix-sub(matrix(((1, 2), (3, 4))),
identity(2)))
```

$$\begin{pmatrix} 0 & 2 \\ 3 & 3 \end{pmatrix}$$

Parameters

```
matrix-sub(
  lhs: bytes content array int float str,
  rhs: bytes content array int float str
) -> bytes
```


lhs bytes or content or array or int or float or str

Left matrix.

rhs bytes or content or array or int or float or str

Right matrix of the same shape.

7.6.7 matrix-mul

Multiply two matrices, or multiply a matrix by a scalar expression.

Matrix multiplication requires compatible inner dimensions. A scalar must be rational-polynomial compatible. Because opaque bytes are interpreted as matrix payloads in the right-hand position, pass scalar atoms as ordinary values or math content rather than as preconstructed atom bytes.

```
#to-typst(matrix-mul(matrix(((1, 2), (3, 4))),
identity(2)))
```

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$$

Parameters

```
matrix-mul(
  lhs: bytes content array int float str,
  rhs: bytes content array int float str
) -> bytes
```

lhs bytes or content or array or int or float or str

Left matrix.

rhs bytes or content or array or int or float or str

Right matrix, or scalar Typst value/math content.

7.6.8 matrix-div-scalar

Divide every matrix entry by a nonzero scalar expression.

The scalar must be rational-polynomial compatible.

```
#to-typst(matrix-div-scalar(matrix(((2, 4), (6,
8))), 2))
```

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$$

Parameters

```
matrix-div-scalar(
  lhs: bytes content array int float str,
  rhs: bytes content int float str
) -> bytes
```

lhs bytes or content or array or int or float or str

Matrix dividend.

rhs bytes or content or int or float or str

Nonzero scalar divisor.

7.6.9 transpose

Transpose a matrix, exchanging rows and columns.

```
#to-tpst(transpose(matrix(((1, 2), (3, 4)))))
```

$$\begin{pmatrix} 1 & 3 \\ 2 & 4 \end{pmatrix}$$

Parameters

`transpose(matrix: bytes content array int float str) -> bytes`

matrix bytes or content or array or int or float or str

Matrix to transpose.

7.6.10 det

Compute the exact determinant of a square matrix.

The result is an atom payload rather than a matrix payload.

```
#to-tpst(det(matrix(((1, 2), (3, 4)))))
```

$$-2$$

Parameters

`det(matrix: bytes content array int float str) -> bytes`

matrix bytes or content or array or int or float or str

Square matrix.

7.6.11 inv

Compute the exact inverse of an invertible square matrix.

Singular and non-square matrices produce an error.

```
#to-tpst(inv(matrix(((1, 2), (3, 4)))))
```

$$\begin{pmatrix} -2 & 1 \\ \frac{3}{2} & -\frac{1}{2} \end{pmatrix}$$

Parameters

`inv(matrix: bytes content array int float str) -> bytes`

matrix bytes or content or array or int or float or str

Invertible square matrix.

7.6.12 matrix-solve

Solve the matrix equation $A \cdot x = b$ exactly.

The row counts of A and b must agree. This strict solver reports an error when the system does not have the required unique solution; use `matrix-solve-any` for underdetermined systems.

```
#let A = matrix($mat(2, 1; 1, -1)$)
#let b = vec((5, 1))
#to-typst(matrix-solve(A, b))
```

$$\begin{pmatrix} 2 \\ 1 \end{pmatrix}$$

Parameters

```
matrix-solve(
  A: bytes content array int float str,
  b: bytes content array int float str
) -> bytes
```

A bytes or content or array or int or float or str

Coefficient matrix.

b bytes or content or array or int or float or str

Right-hand-side matrix or column vector.

7.6.13 matrix-solve-any

Solve $A \cdot x = b$ exactly, choosing one solution if underdetermined.

The row counts of A and b must agree. An inconsistent system still produces an error.

```
#let A = matrix($mat(2, 1; 1, -1)$)
#let b = vec((5, 1))
#to-typst(matrix-solve-any(A, b))
```

$$\begin{pmatrix} 2 \\ 1 \end{pmatrix}$$

Parameters

```
matrix-solve-any(
  A: bytes content array int float str,
  b: bytes content array int float str
) -> bytes
```

A bytes or content or array or int or float or str

Coefficient matrix.

b bytes or content or array or int or float or str

Right-hand-side matrix or column vector.

7.6.14 row-reduce

Row-reduce a matrix exactly using Gaussian elimination.

Returns (matrix: bytes, rank: int). By default every column may contain a pivot. Set max-col to limit pivot search to the first max-col columns, which is useful for an augmented matrix whose trailing columns are right-hand sides; all columns are still transformed by the row operations.

```
#let rr = row-reduce(matrix(((1, 2), (3, 4))))
rank #rr.rank: #to-tyrst(rr.matrix)
```

rank 2: $\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$

Parameters

```
row-reduce(
  matrix: bytes content array int float str,
  max-col: int none
) -> dictionary
```

matrix bytes or content or array or int or float or str

Matrix to reduce.

max-col int or none

Number of leading columns eligible for pivots. none uses every column. An explicit value must be between zero and the column count.

Default: none

7.6.15 augment

Horizontally concatenate two matrices as [lhs rhs].

Both matrices must have the same number of rows.

```
#to-tyrst(augment(identity(2), matrix(((1, 2),
(3, 4)))))
```

$\begin{pmatrix} 1 & 0 & 1 & 2 \\ 0 & 1 & 3 & 4 \end{pmatrix}$

Parameters

```
augment(
  lhs: bytes content array int float str,
  rhs: bytes content array int float str
) -> bytes
```

lhs bytes or content or array or int or float or str

Left block.

rhs bytes or content or array or int or float or str

Right block with the same row count.

7.6.16 split-col

Split a matrix into left and right column blocks.

Returns exactly (left, right), both as matrix payloads. The current matrix backend requires index > 0 and index < columns - 1.

```
#let aug = augment(identity(2), matrix(((1, 2),
(3, 4))))
#let parts = split-col(aug, 2)
#to-tyrst(parts.at(0)) | #to-tyrst(parts.at(1))
```

$$\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \mid \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$$

Parameters

```
split-col(
  matrix: bytes content array int float str,
  index: int
) -> array
```

matrix bytes or content or array or int or float or str

Matrix to split.

index int

Zero-based first column of the right block.

7.6.17 primitive-part

Divide a rational-polynomial matrix by its content.

The result is the primitive matrix whose coefficient GCD has been removed.

```
#let x = var("x")
#let P = matrix(((mul(2, x), mul(4, x)),
(mul(6, x), mul(8, x))))
#to-tyrst(primitive-part(P))
```

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$$

Parameters

```
primitive-part(matrix: bytes content array int float str) -> bytes
```

matrix bytes or content or array or int or float or str

Matrix whose entries are rational polynomials.

7.6.18 content

Compute the coefficient content of a rational-polynomial matrix.

The content is the common coefficient GCD and is returned as an atom payload.

```
#let x = var("x")
#let P = matrix(((mul(2, x), mul(4, x)),
(mul(6, x), mul(8, x))))
#to-tyrst(content(P))
```

$$2x$$

Parameters

```
content(matrix: bytes content array int float str) -> bytes
```

matrix bytes or content or array or int or float or str

Matrix whose entries are rational polynomials.

7.6.19 matrix-at

Read one matrix entry as an atom payload using zero-based indices.

```
#let A = matrix(((1, 2), (3, 4)))
#to-tyrst(matrix-at(A, 0, 1))
```

2

Parameters

```
matrix-at(
  matrix: bytes | content | array | int | float | str,
  row: int,
  col: int
) -> bytes
```

matrix bytes or content or array or int or float or str

Matrix to index.

row int

Zero-based row index. Must be within the matrix bounds.

col int

Zero-based column index. Must be within the matrix bounds.

7.6.20 matrix-shape

Return the matrix shape as the two-element array (rows, columns).

```
#repr(matrix-shape(matrix(((1, 2), (3, 4)))))
```

(2, 2)

Parameters

```
matrix-shape(matrix: bytes | content | array | int | float | str) -> array
```

matrix bytes or content or array or int or float or str

Matrix to inspect.

7.7 Scalar constructors

7.7.1 add

Construct an exact sum from expression values.

Arguments are converted as by atom. With no arguments, the result is the additive identity zero.

```
#to-tyrst(add("x", 1, "y"))
```

$1 + x + y$

Parameters

```
add(..terms: arguments) -> bytes
```

..terms arguments

Positional terms to add.

7.7.2 mul

Construct an exact product from expression values.

Arguments are converted as by atom. With no arguments, the result is the multiplicative identity one.

```
#to-tyrst(mul(2, "x", "y"))
```

 $2xy$

Parameters

```
mul(..factors: arguments) -> bytes
```

..factors arguments

Positional factors to multiply.

7.7.3 neg

Negate an expression exactly.

```
#to-tyrst(neg("x"))
```

 $-x$

Parameters

```
neg(expr: bytes | content | int | float | str) -> bytes
```

expr bytes or content or int or float or str

Expression to negate.

7.7.4 sub

Subtract rhs from lhs exactly.

```
#to-tyrst(sub("x", "y"))
```

 $x - y$

Parameters

```
sub(
  lhs: bytes | content | int | float | str,
  rhs: bytes | content | int | float | str
) -> bytes
```

lhs bytes or content or int or float or str

Minuend.

rhs bytes or content or int or float or str

Subtrahend.

7.7.5 div

Construct the exact quotient lhs / rhs.

```
#to-tyrst(div(1, "x"))
```

$$\frac{1}{x}$$

Parameters

```
div(
  lhs: bytes content int float str,
  rhs: bytes content int float str
) -> bytes
```

lhs bytes or content or int or float or str

Numerator.

rhs bytes or content or int or float or str

Denominator.

7.7.6 pow

Construct the exact power base ^ exp.

```
#to-tyrst(pow("x", 3))
```

$$x^3$$

Parameters

```
pow(
  base: bytes content int float str,
  exp: bytes content int float str
) -> bytes
```

base bytes or content or int or float or str

Base expression.

exp bytes or content or int or float or str

Exponent expression.

8 Compatibility, attribution, and licensing

This manual describes Tymbolica 0.1.0 and is compiled against the bundled `tymbolica.wasm`. The package is tested with Typst 0.14 or newer.

Tymbolica’s original source code is released under the [MIT License](#). Symbolica is developed by the Symbolica contributors and is distributed under its own [license terms](#). The MIT License does not relicense Symbolica or the bundled WebAssembly artifact; Symbolica’s terms still apply to their use.

For source, issues, and release history, visit github.com/lcnbr/tymbolica.